Network Design Decision Report

16/05/2026

LHN

Ken Doan, Kyle Greenwood

Version Control

Version	Date	Author/s	Notes
V1.0	8/4/2026	Ken, Kyle	Document created
V2.0	16/5/2026	Ken, Kyle	Document completed
V2.1	17/10/2026	Ruby	Report formatting updated

Table of Contents

Version Control 2

Table of Contents 3

1. Introduction 5

2. System Requirements 5

3. Initial Recommendations 6

3.1 Network Layout 6

3.2 LoRa 6

3.3 MeshCore 7

4. Hardware Considerations 8

5. Software Considerations 10

6. Network Topology Options 14

6.1 Network Layout 14

6.1.1 Hub and Spoke 14

6.1.2 Tree 14

6.1.3 Full Mesh 15

6.1.4 Clustered Mesh 15

6.2 Sensor Node Implementation 15

6.3 Network Node Placement 15

6.4 Network Resolution 15

6.4.1 Current Bushfire Detection & Monitoring Background 16

6.4.2 Analysis of Existing Systems for Fire Monitoring Context 16

6.4.3 Network Resolution Conclusion 16

6.5 Network Topology Recommendations 17

7. Testing Results & Analysis 18

7.1 Testing Conclusion 19

8. Antenna Type and Configurations 20

9. Cost Analysis 21

10. Integration with Existing Networks 22

11. Risks and Limitations 23

11.1 ACMA Regulatory Compliance 23

11.2 Network Failure Risk Mitigation Strategies 24
11.2.1 Inter-Symbol Interference 24
11.2.2 External Interference 24
11.2.3 Network Flood 24
11.2.4 Packet Loss 24
11.2.5 Node Recovery 25
12. Conclusion 26
13. References 27
14. Appendix 29
14.1 Appendix A: AI response for C, C++ and Rust comparison 29

1. Introduction

The purpose of the network system is to automate the delivery of telemetry data collected from the hazard sensors to a central network that can be accessed by the local community and the SES to process and analyze.

Since the central network for processing the hazard telemetry already exists, the network infrastructure for the hazard sensors will need to interface with this central network. The main protocol being used for said interface is MeshCore, a “secure, decentralised mesh radio platform powered by LoRa”. This requirement was established by the project hosts.

The method of how the telemetry travels from the remote hazard sensors to the gateway node of the central network has been delegated to the project team to design and optimize. The main purpose of this document is to explore feasibility of various designs, features and products for solving the network requirement of the project solution.

2. System Requirements

The system requirements for the network infrastructure are outlined as follows:

- The system shall use MeshCore as the primary protocol for communicating with the central network.
- The system shall minimize power consumption as much as practicable to prolong the duration the system can operate without direct maintenance.
- The system shall use the ISM LoRa band for transmitting all communications between sensors, nodes and repeaters.
- The system shall adhere to all legal regulations for transmitting data within the ISM band.
- The system shall use commercially available off-the-shelf products.
- The system shall be low-cost and replaceable in case equipment are degraded or destroyed through natural events or tampering from wildlife or third-party individuals.
- The system shall be small enough to easily be transported to and installed at remote locations in the NSW bushlands.
- The system shall be operational in outdoor conditions typical of the NSW bushlands.

3. Initial Recommendations

3.1 Network Layout

Through early discussions with the technical advisor for the project, Asanka Peramune, a basic architecture of the system and some pre-selected products were recommended to the project team for the hazard sensor network. Figure 1 shows a diagram of the network layout recommended for this project by Peramune.

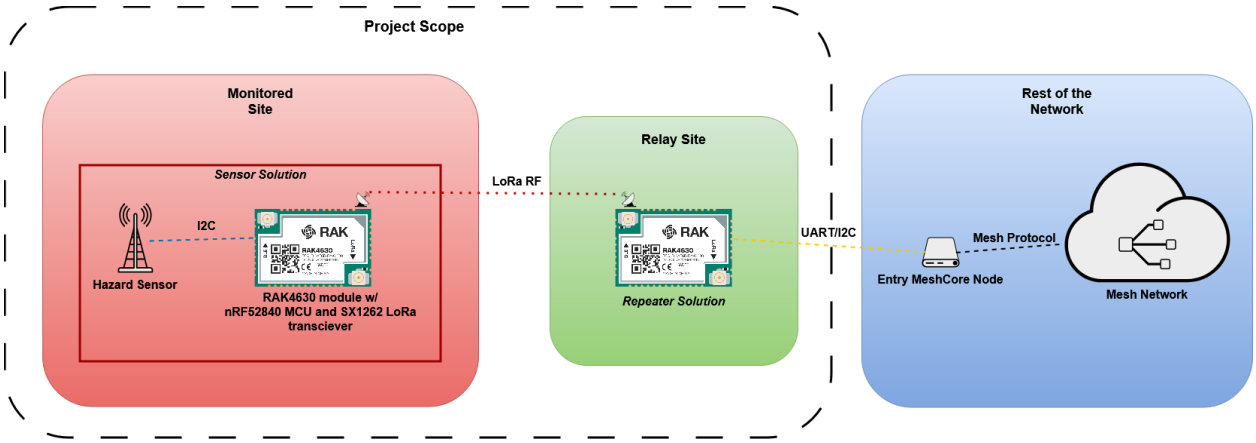


Figure 1. Functional Block Diagram of Hazard Sensor Network

This layout has three distinct sites, the monitored site where hazard sensors are located and data is collected, the relay site where a repeater takes incoming data from the hazard sensors and boosts signal for longer range transmission, and the rest of the network (aka the central network) which receives the incoming data for the community and SES to see.

In addition to providing the recommended architecture, Peramune also recommends the project team to use LoRa and MeshCore as the two main technologies for sending telemetry across the network.

3.2 LoRa

Peramune recommends the use of LoRa as the main transmission technology between the sensor node and the repeater node. LoRa (Long Range) is a low-power wireless modulation technique that is designed for transmitting small, packetized radio streams over long distances. It is commonly used in Internet-of-Things (IoT) devices and telemetry systems where devices need to operate for long periods of time on limited power and transmitting at low bit rates is suitable. Derived from Chirp Spread Spectrum (CSS) technology, LoRa encodes information on radio waves using chirp pulses, resulting in the transmission being robust against disturbances and noise as it travels over great distances. The typical system architecture for a LoRa network is to have a sensor node that collects data and sends it to a gateway using LoRa. The gateway receives the LoRa signals and routes the data to the internet or other network which

then processes and routes the data accordingly. This architecture is the same architecture recommended by Peramune above.

3.3 MeshCore

The last technology recommendation from Peramune is the use of MeshCore as the underlying mesh platform network for our hazard sensor network architecture. It is an open-source mesh radio platform built on LoRa radios, providing a decentralized network topology where a LoRa node can connect directly and non-hierarchically to other surrounding nodes, communicating peer-to-peer to cooperate in routing data throughout the network.

MeshCore is used to dramatically extend the range of a LoRa network, since messages can hop between nodes to get to areas an individual radio would be unable to reach. It does this while also maintaining low power operation, making it suitable for battery-powered devices. Decentralisation allows the network to operate, without the need for gateways or a central server, instead allowing each node to forward data packets making it suitable for off-grid communication. This makes the network resilient, when a node goes down, the surrounding nodes can reroute their data to adjacent nodes which keep the network alive. This is desirable for the hazard sensor network where a node could potentially be taken out due to bushfires, flooding or other environmental hazards. MeshCore comes with a companion app that makes it easier to connect directly to nodes and test transmission and receive at various points in the network.

Previously, Meshtastic was used as the mesh platform for the flood sensor network. However, it faced a major issue in the form of packet loss due to the lack of routing capabilities. Packets were sometimes being routed to the wrong nodes due to the internal protocol of Meshtastic. MeshCore overcomes this limitation by allowing the user to explicitly state which nodes a specific data packet should be routed through to reach its location. This was biggest deciding factor for the project host to switch to using MeshCore technology over Meshtastic. MeshCore's main limitations are its latency due to the multi-hop routing and the complex message handling required on every node to allow data to be routed to its destination successfully.

4. Hardware Considerations

Based on recommendations from Peramune, the project team is considering the RAK4630 module as the core hardware for developing the network solution for the hazard sensors. The RAK4630 module bundles together a Nordic nRF52840 MCU and a Semtech SX1262 LoRa® transceiver into a single package with accessible I/O ports for easy integration into any solution, at the price of $15 USD. The module also has two separate IPEX connectors for Bluetooth 5.0 and LoRaWAN transmissions, allowing for quick connectivity with the Nordic MCU and Semtech transceiver, without the need for an additional custom peripheral PCB. Peripheral interfaces supported by the module includes UART, I2C, GPIO and USB, all common interfaces used by multitudes of embedded systems, making it almost definitely compatible with whichever sensor technology the project team proceeds with. The module's small form factor makes it ideal for housing it in a small package along with the sensors to facilitate portability and ease of install at remote sites.

The module operates using a typical 3.3V power rail, a standard of embedded systems, meaning power regulation and conversion should be relatively simple due to a lot of CotS power supply products are made to be compatible with outputting 3.3V. Additionally, Table 1 shows the module's power consumption at various modes. In Idle mode, the module only draws 3.35 mA at 3.3V which results in an average power consumption of 11mW. If implementing One-Time Sleep, a feature that puts the device into deep sleep until an external trigger (button press, timer, interrupt, etc.) wakes it up, the current draw is a miniscule 4.42 uA, meaning the total power consumptions is only 14.5 uW. Power consumption during transmit and receive are also low, being 244mW and 8mW respectively when operating at 3.6V. The low power consumption makes this module ideal for use in remote sensor nodes where regular maintenance is hard and power generation from solar panels can be halted for up to 2 weeks depending on the weather.

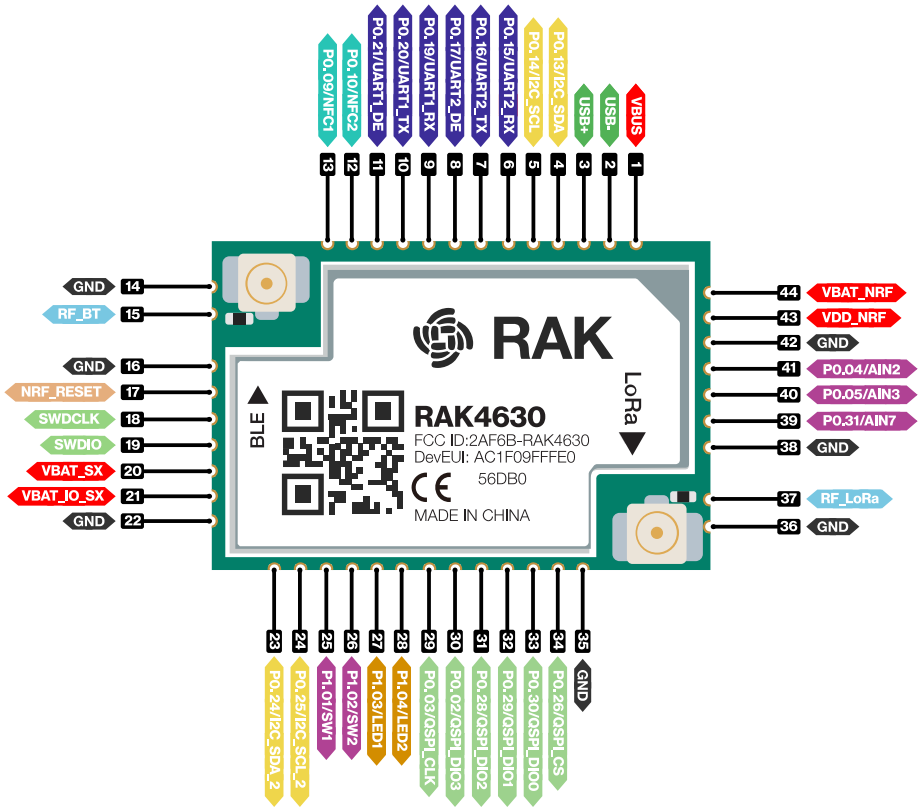


Figure 2. Pinout diagram of the RAK4630 module

Table 1. Power consumption summary from RAK4630 datasheet

Item	Current Average	Condition - Voltage and Dwell Time
RAK4630 Module in One-Time Sleep	4.42 uA	3.3 V 10 seconds
RAK4630 Module System up in Idle mode	3.35 mA	3.3 V 10 seconds
RAK4630 Module in LoRaWAN One-Time Sleep	4.23 uA	3.3 V 10 seconds
RAK4630 Module 15 Bytes Data TX Sending	67.8 mA	3.6 V 1 second (in RAK5005-O board)
RAK4630 Module in RX Window Received	2.22 mA	3.6 V 1 second (in RAK5005-O board)

The RAK4630 is currently being considered by the project team due to it already being used in the development of the repeater network by the project host. As such, there has already been testing done on the hardware by the project to verify that it is suitable in some capacity for the hazard sensor network.

To aid in the integration of the RAK4630 module, the RAK19007 Base Board is considered for interfacing to allow for easier programming, debugging and power supply, since it is designed to be directly compatible with selected RAK module. Noting that the RAK5005-O was replaced by the RAK19007 board.

5. Software Considerations

Included in the scope of the project is the development of custom firmware for network modules to communicate with the sensor, each other, and the gateway node of the central network. Importantly, a language for the development of the code base needed to be decided upon, with the two main contenders being C and Rust. Table 2 shows a comparison of the two languages in the context of embedded development.

Table 2. Comparison between Rust and C for embedded development

Property	**Rust**	**C**
Safety	• Ownership and borrowing system prevent common memory-related bugs like null pointer dereferences or buffer overflow. • Guaranteed memory safety at compile time. • Good for embedded development where memory is a precious resource. • Prevents invalid peripheral usage. • Can carry out async tasks without worrying about race conditions. • Safety is the largest selling point of the language.	• Allows total control of memory. • Easy to make hacks or shortcuts to get the system running. • No safety guarantees and thus requires good coding practices and memory management techniques. • Bugs aren't always caught at compile time and may appear at run-time.
Performance	• Fast performance that is comparable with C/C++. • Zero-cost abstraction (except for when using dynamic dispatch) allows for code to be optimized at compile time, leading to more efficient run-time.	• Maximum low-level control allows for maximum efficiency. • Fastest programming language for embedded development. • Can allow for more optimization if you know what you are doing. • Small caveat that a lot of vendor HALs are bloated and inefficient.
Concurrency	• Uses async "tasks" which allows for lightweight concurrency. • No Real Time Operating System (RTOS) needed for most cases.	• Has mature RTOS support for most MCUs. • Guarantees predictable and consistent response time for tasks regardless of the workload.

	• Strong type system prevents race conditions. • *Fearless Concurrency* – an attribute of Rust's ownership and type system that allows for concurrency bugs to be caught while writing the code, rather than at runtime. • Extremely complex concurrency systems with a lot of features that most users won't need. • Async ecosystem is still being developed. Some crates (Rust code libraries) are still developing their async architecture and functionalities.	• If MCU does not have RTOS support, concurrency can be messy. • RTOS adds memory overhead and extra complexity. • Possible to get race conditions if not careful with task timings.
Portability	• Refactoring is easy. • Trait abstraction allows for easier code integration and computability. • Hardware agnostic when using certain crates such as 'embedded-hal'.	• Refactoring is hard. • Tight integration with specific hardware allows for more vendor features but also causes vendor lock-in. • Firmware is specific to the hardware platform. Moving platforms requires rewriting the code. • HAL APIs are not standardized and vary greatly between vendors.
Readability	• Rust is written to be more readble and less error-prone than C/C++. • Pattern matching, type inference and functional programming constructs makes writing and maintaining code much easier for complex projects. • Generics and heavy abstraction can make it more confusing if not used to the concepts.	• Simple syntax with relatively few keywords. • Lack of abstraction making large project repetitive and hard to organize. • Heavy use of pointers and pointer arithmetic which can be confusing to read. • Manual memory management makes program flow harder to follow.

Learning Curve	• Complex concurrency system can be hard to architecture. • Ownership, borrowing and lifetime concepts are unique to Rust and requires a steep learning curve.	• Already taught to most engineers. • Much easier to start a project. • Much faster to get a project working, runtime bugs aside. • Debugging is usually much harder and more time-consuming to due to the lack of safety features leading to more hidden bugs.
Ecosystem	• Rapidly growing with strong community momentum. • More developers are rewriting their code in Rust and thus developing new crates to aid this. • Embassy is an existing library that provides an excellent framework for modern embedded development. It provides most of what embedded developers need. • Tooling and debuggers are less sophisticated. • Still not as extensive as other languages. • With adoption growing, could be better in the long term.	• Been around for a long time and thus has a lot of pre-existing libraries and solutions for embedded development. • Much larger ecosystem with a lot of official vendor support. • Current industry standard for embedded development. • Complete coverage of all device drivers. • Quality between drivers and libraries vary greatly. • The allowance of shortcuts allows for poor API designs sometimes. • Heavy reliance on vendor code generators (e.g. CubeMX) which are sometimes inefficient..
Compiler	• Much longer compile time compared to C due to the extra safety features and zero-cost abstraction.	• Fast compile time since it forgoes and a lot of code checks compared to Rust.
Community and Support	• New language and so community and support is still growing. • Range of crates are still not as extensive as other languages.	• Language has been around for a long time and so the community is large. • Supported by practically all vendors. • Large catalogue of libraries and community solutions.

Low-level control	• Almost always working at higher abstraction layers, rather than with memory and registers directly. • User is safeguarded by default from interacting with low-level hardware. • Can work at the register level using 'unsafe' functions but is not recommended.	• Direct hardware access allows for full control over memory, flags, registers and protocols. • No abstraction penalties. • Manual allocation and layout of memory allows for fine-tuned control. • Can be extremely unsafe if not handled carefully. • Small mistakes in low-level code can cause major issues. • No safeguards.

With the considerations above, the project team has decided on using Rust to write the firmware for the network devices. One of the team members already has experience in using Rust for embedded development, this weighed heavily on the decision. Although Rust has a steeper learning curve compared to C, the various safeguards it provides to the developer, as well as being able to guarantee no runtime bugs resulting from mismanaged memory or pointers, makes it ideal for building robust and complex embedded projects [7] [8]. Its readability also makes it easier to manage the codebase as it grows more complex in the future. While the community and support may not be as extensive as C, there are already plenty of projects, such as Embassy, that offers enough functionality for full-fledged embedded development. The portability of Rust code using the 'embedded_hal' and 'embassy' code libraries is also a main attractor for the language as it makes the firmware platform agnostic. This means that the if the project was to switch to a different hardware module (i.e. a different Nordic MCU or a different vendor entirely) the firmware can be reused with minimal rewriting of the code.

The main disadvantage of using Rust is that ANU currently only teaches C as part of its curriculum. If this project is to be continued by a future group of ANU students or even the project host, they would also have the challenge of first learning Rust to understand the firmware code. It should be noted that the project host has confirmed that any language was fine to use for the firmware in a previous meeting with the project team.

6. Network Topology Options

6.1 Network Layout

A key consideration in a wide-area network (WAN) design is the topology of the network. Due to the irregularity in terrain across the network and conditions over time, there is no single option that will solve all problems. The options outlined will be assessed in terms of power efficiency (average hop count), node redundancy, and coverage efficiency.

Table 3. Topology Parameter Definitions

Term	Definition
Power efficiency	How many transmissions (hops) will need to be made to reach the central network
Node redundancy	How vulnerable is the network to a single point of failure (SPOF)
Coverage Efficiency	How much area is covered per node

6.1.1 Hub and Spoke

Multiple sensors to one repeater; repeater links to other repeaters (as a pipeline) with only a few hubs.

- Allows for less repeaters per sensor node
- Promotes clustered design; useful for monitoring relatively small regions (with radius from repeater to sensor nodes equal to acceptable link distances).
- Simple routing requirements.
- High risk due to the single point of failure (hub) for each cluster.
- Low average hop count
- Mid level scalability
- Low node redundancy
- High node efficiency

6.1.2 Tree

Repeaters link to multiple other repeaters further into the network; branching with a few local sensors per repeater.

- Balance between the number of sensor and repeater nodes.
- No overloaded repeater nodes.
- Simple routing requirements.
- Reasonably high risk due to the branching nature of network, downstream nodes are at exponentially higher risk.
- Mid average hop count.
- High scalability.
- Reasonably low node redundancy.

6.1.3 Full Mesh

Sensor nodes link to other sensor nodes as well as repeaters with uniform network distribution.

- Maximum node redundancy.
- Limits applicable deployment locations drastically.
- Not directly compatible with sensor and repeater node design.
- Poor scalability.
- Poor coverage efficiency.

6.1.4 Clustered Mesh

Mesh between repeaters where feasible, flexible to pipeline if needed. Partial mesh of sensor nodes around repeaters.

- Complex routing requirements.
- Significant node redundancy.
- Highest deployment flexibility.
- Highest deployment complexity.

6.2 Sensor Node Implementation

Due to the low power requirements at the sensor nodes, the communication between the sensor and repeater nodes will likely be a direct RF connection, instead of using the MeshCore protocol. This saves computational overhead at the sensor, providing higher power efficiency. The drawbacks of this design decision are that the sensor nodes will not be able to mesh reducing the capacity to send node information.

6.3 Network Node Placement

The general orientation of groups of repeater and sensor nodes is an idealised view of the system without proving that a network of a given layout can be deployed in the field. To combat this, a member of the team will develop a tool to give to the client at handover. This network analysis tool is being developed to determine whether a set of proposed node placements is appropriate and assist in finding possible node placement configurations. The development of the network analysis tool is a stretch goal to be developed concurrently through the next phases of the project.

6.4 Network Resolution

The required network resolution is hazard dependent. A flood monitoring network needs a low network resolution and is generally one dimensional (along a river system). While a fire monitoring network sensing for air quality requires a completely different resolution. As fire monitoring is a focus of this project, the network resolution for that implementation will be investigated here. If further into the project timeline it is

confirmed other hazards will be monitored for; the ideal network resolution will have to be re-examined.

6.4.1 Current Bushfire Detection & Monitoring Background

A portion of Eastern Australia's current bushfire detection capability outside of manual reports is described in the data from the Himawari-8 fire detection reports [15]. This detection model focuses on hotspots that indicate either existing fires or high-risk areas. There are errors with detection however, factors also impacting this project can lead to detection errors and therefore error is not considered in this analysis. The analysis will be conducted for two separate approaches, detection of high-risk areas and detection of existing fires. It should be noted that the team will likely soon liaise with FBANs (Fire behavioral analysts) working at CSIRO and some better information on current bushfire detection/monitoring may become available to the team in which case this analysis can be redone with updated data.

6.4.2 Analysis of Existing Systems for Fire Monitoring Context

The rate of bushfire spreading is assumed to be 20km/h, toward the higher end of average bushfire speeds. Detection resolution is assumed to be $2km^2$ every 10 minutes from the active fire detection report. Converting units to make the values comparable gives;

Rate of spread: 20km/h into km/min = 0.33 km/min

Rate of detection: $2km^2/10min = 20km^2/min$

Therefore, the worst-case scenario fires will reach approximately a diameter of 5 km^2 before being detected by the existing system that uses Himawari-8's imaging data.

NOTE: Depending on information provided from FBANs and outcomes of the sensor workstream report, this section may need to be redone. Wade mentioned satellite detection is currently every 3 hours which does not align with this.

6.4.3 Network Resolution Conclusion

Therefore, for there to be any improvement in fire detection provided by this project, the network would have to have detection nodes every $5km^2$ or less. Using the area of the Mandagery (Eugowra) as a case study. The region spans $2004.20km^2$. To implement fire detection in this area using the proposed network:

Increased detection ratio = 5/S = 1.25, S = selected resolution ($4km^2$)

Minimum units required = 2004/4 = 501 units

Cost of network* = 501*500 = 250,500 AUD

*Excluding deployment and maintenance costs

While this may be a reasonable cost for a large-scale deployment, for fiscal efficiency a significantly more targeted deployment to high-risk sub regions is advised.

6.5 Network Topology Recommendations

The network layout will have two layers; a hub and spoke model between repeaters in regions of interest and their associated sensor nodes (layer 1), as well as partial mesh model between repeater nodes (layer 2).

Due to firmware choices, each sensor node in layer 1 will only communicate with its nearest repeater node. This design allows us to exploit the benefits of directional antenna gain, where it would otherwise be inappropriate in a mesh context. Specifically, each sensor node (at the spoke), will have a directional antenna aimed at its repeater, where the repeater receives using an omni-directional antenna, allowing the node to receive from all nearby sensor nodes. This configuration gives maximum power efficiency for the sensor nodes but places a higher overhead on the repeater nodes that must do additional processing.

The repeater nodes will package data into appropriate payloads for MeshCore packets and handle any routing processing. Due to the repeaters being the backbone of the network, they will likely have to make long links from urban areas to a region of interest. In these links there will be a chain of repeaters, with little to no meshing. However, within the region of interest repeater node density should increase to allow for meshing where possible. This is the partial mesh approach that provides a high level of robustness to the network, as well as a high level of deployment flexibility.

The resolution of sensor nodes and repeater nodes will be examined in the integration phase of the projects workstream approach as it is determined by the sensor implementation.

7. Testing Results & Analysis

Lead by the project hosts, testing on the limits of LoRa transmission distance under different parameters was done on 15/4/2026. The testing had four phases that were conducted at two separate sets of testing sites, approximately 67km apart:

1. Testing a LoRa transmitter and receiver, with directional antennas at either testing site.
2. Testing a LoRa transmitter and receiver, with omni-directional antennas at either testing site.
3. Testing MeshCore and a directional (14dBi yagi-uda) antenna at either testing site.
4. Testing MeshCore and an omni-directional antenna at either site.

Table 4. LoRa radio testing results – phase 1 and 2 (site 1).

SF	RSSI (dBm)	SNR (dB)	Observations	Antenna
8	-110	-10	Signal reception intermittent.	Directional
9	-107	-6	Signal reception intermittent.	
10	-101	-9	Signal reception is more frequent than SF 8 & 9.	
11	-97 to -110	-9	Signal reception is more frequent than SF 10.	
12	-97 to -110	-9	Signal reception is more frequent than SF 10.	
12	-106 to -111	-14 to -20	Signal reception is less frequent than directional reception at SF 8.	Omni-Directional

The results of phase 2 testing for all other SF values were the same. A signal was received, but the data was corrupted, therefore either the signal was just noise, or the signal had been sufficiently corrupted that it could not be demodulated and was therefore unusable.

Both phase 3 and 4 had only failed transmissions at site one. Once these phases had been attempted, both teams moved to the second set of testing site locations and went through the testing phases again.

Table 5. LoRa radio testing results – phase 1 and 2 (site 2).

SF	RSSI (dBm)	SNR (dB)	Observations	Antenna
7	-100	6	Consistent reception, outstanding SNR.	Directional
7	-105 to -110	-2	Poor reception consistency.	Omni-Directional
8	-107	-3	Poor reception consistency.	
9	-109	-5	Intermittent reception, more semi-successful transmissions (Rx message corrupt).	

10	-105	-4	Better reception consistency than SF 9.	
11	-109	-4	Better reception consistency than SF 9.	
12	-108	-4	Better reception consistency than SF 9.	

Due to the significant increase in reception quality from site one to site two, most tests here used the omni-directional antenna as the radio has limited functionality at the previous site. There are two notable differences between each set of sites. The elevation at the second site on Mt. Ainslie had a higher elevation of approximately ten meters, leading to better line of sight and less obstructions. Additionally, the first Mt. Ainslie site was directly in front of a minor telecommunication tower. This seemed to have caused interference with the transmissions which is not surprising due to the transmission power differences.

7.1 Testing Conclusion

From the small dataset acquired from testing, increasing SF showed diminishing gains in SNR and RSSI but generally better reception consistency. It can also be concluded that if environmental noise is managed correctly, both the omni-directional and directional antennas are suitable for use in the local hazard network. Link distance must be carefully chosen with an upper bound of 67km based on testing.

8. Antenna Type and Configurations

From the testing conducted, both omni-directional and directional antennas are useable in the network design, both with different strengths and weaknesses as outlined in Table 6.

Table 6. Comparison of Directional and Omni-Directional Antennas.

	Directional	**Omni-Directional**
Benefits	• Significantly higher SNR. • Higher reliability. • More customisability with isotropic gain.	• Less units are required. • Non-directional links give higher network flexibility. • Lower hardware costs. • Easy deployment.
Drawbacks	• More complicated deployment requirements. • Higher cost. • Requires multiple antennas per node.	• Lower SNR, particularly in high interference environments. • Lower reliability.

The network has three distinct options in antenna configuration, purely directional, omni-directional, or a combination of both. If using the hub and spoke or clustered mesh, omni-directional antennas should be used at the spokes due to have the transmission from multiple nodes be received on a single antenna, instead of the multiple directional antennas that would be required for this design.

If either the purely directional antenna or the combined configuration is used, there are three major design considerations. First, a digitally controlled SPDT RF [14] is required due to the RAK boards' single coaxial connection. From this, custom firmware would be required to manage the antenna selection. The control system could be implemented in the routing algorithm, but it would increase overall system complexity and the scope of work. The last design consideration is a significant technical limitation; since only a single antenna can be selected at a time, only a single antenna can receive data at a time. This restriction essentially forces the system to be listening to the downstream sensor nodes (as well as downstream repeater nodes) as a default or upstream allowing for continuous node management for the repeater. This restriction makes the use of directional antennas a far less feasible option but should still be considered due to the significant signal reception benefits.

9. Cost Analysis

The cost of a RAK4630 module is $15USD, and to program it a RAK19007 base board is required which costs, $9.99USD. Additionally, omni-directional antennas for BLE and LoRa transmission costs $3USD each. Directional antennas range from $3USD to $16USD depending on isotropic gain. Therefore, the total hardware cost for developing a single unit of the LoRa transceiver starts at $30.99USD, not including the cost of the chassis to house it. However, the final hardware cost depends on the chosen configuration, with an upper cost of $43.99USD. This is inexpensive, relative to the microgrant budget of $150AUD granted to the project team by the ANU. Some costs can be saved when producing more units by reusing the RAK19007 base board to program multiple RAK4630 modules. This is also cheap enough to not be concerned with modules being destroyed or damaged by natural events or intentional tampering when deployed in remote outdoor locations.

10. Integration with Existing Networks

In the region of interest, there is an existing Meshtastic network. However, to bridge between the Meshtastic and MeshCore protocols would require a significant scope of work. In addition, it would be inefficient as MeshCore has been chosen for this project specifically to overcome the packet loss experienced in Meshtastic. This considered integrating with the existing Meshtastic network is not feasible and can be left outside the scope of the project. Therefore, to integrate to an existing network requires a MeshCore network to be accessible in the region of interest, where there currently is not. As MeshCore is an emerging technology, uptake in the scoped region will be monitored (via resources [6] and [9]) to reconsider the possibility of integration.

If integration is to take place further into the project, carrier frequency, bandwidth, and spreading factor must match the existing network. If the local hazard network requires different parameters for any reason at the point of integration, there can be a gateway node that bridges between the two configurations.

11. Risks and Limitations

11.1 ACMA Regulatory Compliance

ACMA is the regulatory body that governs radio frequency (RF) communications in Australia among other similar fields. ACMA defines the allowable operation of transmission across the RF spectrum, dividing it into frequency bands with separate regulations. For this project, the systems designed will fall under the Low Interference Potential Devices (LIPD) class of radiocommunication devices defined by ACMA. The LIPD class license is an instrument that authorizes and outlines the use of LIPD devices without the need for individual licenses. The operating parameters outlined in the *LIPD Class License 2025* [5] document are; Permitted operating frequency band, maximum effective isotropic radiated power (EIRP), class of transmitter, and any additional limitations. Under this license there are two bands that are compatible with MeshCore and comply with regulations;

Band 1:	433.05-434.79MHz	(Item 19, Table 1, [5])
Band 2:	915-928MHz	(Item 20, Table 1, [5]), (Item 1, Table 8, [5])

Band 1 has a maximum EIRP of 25mW (~13.98dBm), with no additional limitations. Band 2 has a maximum EIRP of 3mW referenced in the G*eneral table* (Table 1). However, in the *Frequency hopping, WiFi and RLAN transmitters* table (Table 8), band 2 is referenced again for two separate class of radiocommunications transmitters; frequency hopping transmitters and digital modulation transmitters. Both transmitter classes have additional limitations. For frequency hopping transmitters, fewer than 20 hopping frequencies must be used during operation (Clause 43). The specific implementation of Chirp Spread-Spectrum (CSS) that LoRa uses has a number of hopping frequencies directly determined by the spreading factor (SF). The maximum SF for standard LoRa is 12, which is within the 20 frequency limit, allowing the use of LoRa transmission devices.

Due to the lower frequency of band 1 there is a significant increase in effective distances transmitters in this range can reach due to the approximately inverse proportionality between frequency and effective signal distances. This factor makes band 1 worth considering as a secondary option even though maximum permitted transmission power is a quarter of that of band 2. For band 1 to be useable, a signal with maximum spreading factor must be able to be sent without falling outside of the allowed frequency band. Band 1 has a bandwidth of 1.74MHz, assuming a SF of 12 is required, the maximum bandwidth for LoRa that could be used is 125kHz. This results in the use of 1.5MHz of the available bandwidth. While 125kHz is the standard bandwidth for LoRaWAN, for this project a higher bandwidth may be more desirable with MeshCore.

Due to the power and frequency limitations outlines in this section, it is recommended that only the 915-928MHz band should be used for this project. Noting that the maximum EIRP of 1W refers to the signal power leaving the antenna. This is not a limitation but a design consideration that improves efficiency when using an antenna with isotropic gain such as the yagi-uda directional antennas.

11.2 Network Failure Risk Mitigation Strategies

General network design principles apply to this project such as managing inter-symbol interference (ISI), external interference, network flood, and packet loss. In addition, node recovery is required which is specific to a remotely maintained internet of everything (IoT) based networks. Below is a breakdown of how each risk is mitigated.

11.2.1 Inter-Symbol Interference

The LoRa modulation scheme is Chirp Spread-Spectrum (CSS), which by design sends signals that are quasi-orthogonal. This significantly reduces ISI but does not eliminate it. To bridge the gap between quasi and fully orthogonal high-power computations must be made which is not feasible due to the low power requirements of the network's nodes. If directional antennas are used, this will further reduce ISI.

11.2.2 External Interference

The data from the network must reach a local community or broker to be useable. This means approaching potentially RF rich environments in which the low power LoRa signals could be lost or corrupted. To mitigate this risk, the choice of node placement within a community is important. Initially desirable locations such as community halls or fire brigades may not be suitable as they are often the most RF dense locations in rural areas. In addition to placing nodes away from existing high-power transmitters, directional antennas produce signals with far better SNR in urban environments as shown in the testing results. Therefore, for the links between community centers and the rest of the network, directional antennas on the receiver side are preferable.

11.2.3 Network Flood

MeshCore has specifically been chosen over Meshtastic to combat this risk with its more sophisticated routing. However, further improvements can be made to reduce the risk of network flood. The sensor nodes of the network will be designed to send data at an offset to each other, which leads to lower network load. This combined with custom routing specifications will sufficiently reduce the risk of network flood.

11.2.4 Packet Loss

The previous network failure mitigation strategies are all focused around reducing the chances of packet loss. This section refers to how to address packet loss when it occurs. When a transmission fails due to packet loss, the transmitter automatically

attempts to retransmit. The parameters around retransmission such as frequency and number of attempts before stopping can be manually configured.

11.2.5 Node Recovery

It is inevitable that a node will be lost somewhere in the network due to any number of possible reasons. To reduce network maintenance overhead, nodes must be manually recoverable. This capability will also provide more information about node status both indirectly and directly. The simplest solution is using the MeshCore MQTT branch of firmware [10] that provides a power management module [11] including voltage monitoring on nRF52 baseboards. This directly provides node health information for large scale network health monitoring (node power levels, charging cycles, etc.). In addition, when multiple attempts at node recovery are made in good conditions (sufficient sunlight has been available to recharge the node over time), it is indirectly given through deduction that there is some sort of hardware failure at the node.

12. Conclusion

The RAK4630, mounted on a RAK19007 baseboard, will be used as the transceiver for all nodes. The firmware for each node will be developed in Rust. The network will be separated into two layers, a mesh network of repeater nodes. In addition to small hub and spoke networks centered around repeaters in key locations, surrounded by sensor nodes at the spokes. Sensor nodes will use directional antennas in higher interface environments and longer links; omni-directional antennas will be used for all other transceivers.

To allow for easier access to node health telemetry, the MQTT MeshCore library will be used. The technical specifications for all transmissions in the network will be as defined in Table 7, noting that the transmission power from transceiver to antenna needs to be lowered to not exceed the EIRP limitations. Code rate and bandwidth are to be determined later and maximized if the power subsystem can sustain a higher draw.

Table 7. Transmission Parameters

Bandwidth (MHz)	**Spreading Factor**	**Transmission Power (W)**
915-928	12	1 (EIRP)

13. References

[1] LoRa Alliance Technical committee, "LoRaWAN™ Regional Parameters," Jun. 2016. Accessed: Oct. 17, 2026. [Online]. Available: https://lora-alliance.org/wp-content/uploads/2020/11/lorawan_regional_parameters_v1_0-20161012_1397_1.pdf

[2] A. YEGIN and N. SORNIN, "LoRaWAN™ 1.0.3 Specification," Jun. 2018. Accessed: Oct. 17, 2026. [Online]. Available: https://lora-alliance.org/wp-content/uploads/2020/11/lorawan1.0.3.pdf

[3] ACMA, "ACMA-mandated Electromagnetic Compatibility (EMC) Standards," 2019. Accessed: Oct. 17, 2026. [Online]. Available: https://www.acma.gov.au/sites/default/files/2019-08/ACMA-mandated%20EMC%20standards.pdf

[4] ACMA, "Australian Radiofrequency Spectrum Plan (2025 Update) 2021." Accessed: Oct. 17, 2026. [Online]. Available: https://www.acma.gov.au/sites/default/files/2026-03/Australian%20Radiofrequency%20Spectrum%20Plan%20%282025%20Update%29%202021_Including%20general%20information.pdf

[5] Department of Infrastructure, Transport, Regional Development, Communications, Sport and the Arts, "Radiocommunications (Low Interference Potential Devices) Class Licence 2025 - Federal Register of Legislation," Legislation.gov.au. Accessed: Oct. 17, 2026. [Online]. Available: https://www.legislation.gov.au/F2025L01047/asmade/text

[6] CARTO, "Aus Eastern Live Mesh Map," OpenStreetMap. Accessed: Oct. 17, 2026. [Online]. Available: https://map.eastmesh.au/

[7] J. Lee, "Rust + Embedded: A Development Power Duo," Developer Portal. Accessed: Oct. 17, 2026. [Online]. Available: https://developer.espressif.com/blog/rust-embedded-a-development-power-duo/

[8] P. Wayner, "7 Reasons to Love the Rust Language—and 7 Reasons Not To," InfoWorld. Accessed: Oct. 17, 2026. [Online]. Available:

https://www.infoworld.com/article/2336934/7-reasons-to-love-the-rust-language-and-7-reasons-not-to.html

[9] OpenStreetMap, “Network Map,” Meshcore. Accessed: Oct. 17, 2026. [Online]. Available: https://meshcore.co.uk/map.html

[10] “Meshcore-MQTT Firmware,” Github. [Online]. Available: https://github.com/Colorado-Mesh/MeshCore-MQTT-Firmware/tree/dev

[11] “nRF52 Power Management Module,” Github. [Online]. Available: https://github.com/Colorado-Mesh/MeshCore-MQTT-Firmware/blob/dev/docs/nrf52_power_management.md

[12] “9dBi 915MHz Yagi Antenna for LoRa NB-IoT N-Female,” aliexpress. Accessed: Oct. 17, 2026. [Online]. Available: https://www.aliexpress.com/item/3256809383260294.html?src=ibdm_d03p0558e02r02&sk=&aff_platform=google&aff_trace_key=&af=&cv=&cn=&dp=&aff_short_key=UneMJZVf

[13] “ZQTMAX 9 Unit 13dB Yagi Antenna for Cell Phone Signal Booster 800 / 850 / 900 MHz LTE CDMA GSM Repeater 824-960MHz 50CM - AliExpress 509,” aliexpress. Accessed: Oct. 17, 2026. [Online]. Available: https://www.aliexpress.com/item/3256801382236700.html?src=ibdm_d03p0558e02r02&sk=&aff_platform=google&aff_trace_key=&af=&cv=&cn=&dp=&aff_short_key=UneMJZVf

[14] “PE42020 pSemi,” RFMW. [Online]. Available: https://www.rfmw.com/products/detail/pe42020-psemi/554111/

[15] A. P. K. Reinke, P. S. Jones, and D. C. Engel, “ACTIVE FIRE DETECTION USING THE HIMAWARI-8 SATELLITE,” BNHCRC. [Online]. Available: https://naturalhazards.com.au/crc-collection/downloads/active_fire_detection_using_the_himawari-8_satellite_final_project_report.pdf

14. Appendix

14.1 Appendix A: AI response for C, C++ and Rust comparison

Comparison of C, C++, and Rust for Embedded Development

1. Overview of Each Language

C

- Procedural, low-level language
- Direct hardware control
- Industry standard for embedded firmware

C++

- Extension of C with object-oriented and generic programming
- Supports higher-level abstractions
- Widely used in complex embedded systems

Rust

- Modern systems language
- Focus on memory safety and concurrency
- No garbage collector, designed for performance

---

2. Performance and Resource Usage

C

- Extremely efficient
- Minimal abstraction overhead
- Predictable performance

C++

- Comparable to C *if used carefully*
- Abstractions can introduce overhead if misused
- “Zero-cost abstractions” possible, but not guaranteed

Rust

- Comparable to C/C++
- Guarantees zero-cost abstractions by design
- Slightly larger binaries in some cases

Verdict:

- All three can achieve near-identical performance
- C is the most predictable; Rust enforces efficiency more strictly than C++

---

3. Memory Management

C

- Manual memory management
- High risk of:
    - Buffer overflows
    - Memory leaks
    - Undefined behavior

C++

- Manual + RAII (Resource Acquisition Is Initialization)
- Safer than C if using:
    - Smart pointers
    - STL containers
- Still possible to write unsafe code

Rust

- Ownership model enforces:
    - No null pointers
    - No data races
    - No use-after-free
- Memory safety guaranteed at compile time

Verdict:

- Safest: Rust
- Balanced: C++
- Riskiest: C

---

4. Real-Time Capability

C

- Excellent determinism
- Standard in RTOS and bare-metal systems

C++

- Also excellent

- Deterministic if avoiding:
    - Exceptions
    - Dynamic allocation in critical paths

Rust

- Strong real-time support
- No garbage collection
- Predictable execution

Verdict:

- All three are suitable for real-time systems
- C and Rust are more naturally predictable; C++ requires discipline

---

5. Hardware Access and Low-Level Control

C

- Direct register manipulation
- Maximum control over hardware
- Vendor SDK standard

C++

- Same low-level access as C
- Can wrap hardware in safer abstractions (classes, templates)

Rust

- Access via safe/unsafe boundaries
- Uses crates like embedded HALs
- Slightly more complex to interface with raw hardware

Verdict:

- Most direct: C
- Best abstraction potential: C++
- Safest structured access: Rust

---

6. Code Complexity and Maintainability

C

- Simple language
- Large projects become hard to maintain

- No built-in abstraction mechanisms

C++

- Highly expressive
- Supports:
    - Object-oriented design
    - Templates
- Can become overly complex (“too many ways to do things”)

Rust

- Enforces clean architecture
- Strict compiler prevents bad patterns
- Steep learning curve, but leads to maintainable code

Verdict:

- Best maintainability (long-term): Rust ≈ C++
- Simplest (but less scalable): C

---

7. Ecosystem and Industry Support

C

- Massive ecosystem
- All microcontrollers supported
- Vendor libraries primarily written in C

C++

- Strong ecosystem
- Supported by many embedded frameworks
- Used in platforms like Arduino and more advanced systems

Rust

- Growing ecosystem
- Not all chips fully supported
- Community-driven development

Verdict:

- Best support: C
- Strong: C++
- Emerging: Rust

---

8. Safety and Reliability

C

- No built-in safety
- Relies entirely on developer discipline

C++

- Safer than C if modern practices used
- Still allows unsafe patterns

Rust

- Enforces safety at compile time
- Eliminates entire classes of bugs

Verdict:

- Best: Rust
- Moderate: C++
- Weakest: C

---

9. Learning Curve and Developer Productivity

C

- Easy to learn basics
- Hard to master safely

C++

- Complex language
- Many features and paradigms
- Can be overwhelming

Rust

- Steep initial learning curve
- Compiler is strict but helpful
- Productivity improves after mastery

Verdict:

- Easiest start: C
- Most complex: C++
- Hardest initially but most guided: Rust

---

10. Power Efficiency

All three:

- Compile to native code
- No runtime overhead (if used properly)

Verdict:

- Essentially equal (depends on implementation, not language)

---

11. Typical Use Cases in Embedded Systems

C

- Bare-metal firmware
- Device drivers
- RTOS kernels
- Low-end microcontrollers

C++

- Complex embedded systems
- Middleware and application layers
- Systems requiring abstraction (e.g., robotics, automotive)

Rust

- Safety-critical systems
- New embedded projects
- Systems where reliability and concurrency matter

---

12. Summary Table

Feature	C	C++	Rust
Performance	Excellent	Excellent	Excellent
Memory Safety	Low	Medium	Very High
Real-Time Support	Excellent	Excellent	Excellent
Ease of Use	Medium	Low	Low (initial)
Maintainability	Low	High	High

Feature	C	C++	Rust
Ecosystem	Very Mature	Mature	Growing
Hardware Control	Direct	Direct + abstraction	Structured

---

13. Practical Recommendation (for your LoRaWAN telemetry project)

- Use **C** if:
  - You need maximum compatibility with LoRa modules and vendor SDKs
  - You're targeting very constrained hardware
- Use **C++** if:
  - You want better structure and scalability
  - Your system is moderately complex (multiple modules, drivers, logic layers)
- Use **Rust** if:
  - You want high reliability and memory safety
  - You're willing to work with a **less mature ecosystem**

Bottom line

- C = control and compatibility
- C++ = structure and scalability
- Rust = safety and modern reliability

If this is for your feasibility report, I can tighten this into a formal comparison section with justification tied directly to your chosen hardware platform.